

JAMES DE RAJA

Unity Rendering & XR Performance Engineer | Frame Pacing

Email: jamesderaja@gmail.com | Location: Open to Remote/Hybrid

Portfolio: <https://james.alphaden.club> | GitHub: <https://github.com/JamesDeRaja> | XR Lab Repo:

<https://github.com/JamesDeRaja/XRPerformanceLab> | LinkedIn: <https://www.linkedin.com/in/james-de-raja>

SUMMARY

Unity Rendering Performance Engineer with 10+ years in real-time systems, specializing in GPU/CPU bottleneck analysis, rendering optimization, and frame pacing under 72/90Hz frame deadlines.

Built deterministic Unity 6 URP + OpenXR profiling workflows to isolate overdraw, MSAA amplification, draw calls, and render thread submission overhead with evidence.

Shipped mobile and XR performance-critical work with a repeatable CPU/GPU profiling mindset using Unity Profiler and Frame Debugger.

CORE SKILLS (ATS KEYWORDS)

- Unity Rendering Pipelines (URP, Built-in)
- Real-Time Performance Optimization
- GPU / CPU Bottleneck Analysis
- Profiling & Diagnostics (Unity Profiler, Frame Debugger)
- XR Frame Timing & Frame Pacing (72/90Hz)
- Memory & GC Optimization (C#, Unity)

Tools / Technical: Unity 6, URP, OpenXR, RenderDoc, Android Profiling, Xcode Instruments, Git

Keywords: XR performance, frame pacing, frame timing, rendering optimization, overdraw, MSAA, draw calls, batching/instancing, CPU main thread, render thread, GPU RenderLoop, profiling, performance instrumentation.

SELECTED PERFORMANCE ENGINEERING PROJECTS

XR Performance Stress Lab (Unity 6 URP + OpenXR)

- Built deterministic rendering benchmarks to isolate overdraw, MSAA bandwidth cost, submission overhead, and frame pacing variance in XR.
- Measured +7.27ms GPU RenderLoop growth from layered transparency and validated root cause with Unity Profiler + Frame Debugger evidence.
- Validated mitigation options for draw calls, instancing / batching, and transparent pass pressure to protect frame timing budgets.

Tech: Unity 6, URP, OpenXR, Unity Profiler, Frame Debugger

Instancing vs Non-Instancing Lab (Submission Cost Isolation)

- Diagnosed render thread pressure in high draw-call scenes via CPU/GPU profiling and controlled A/B scene conditions.
- Improved CPU submission behavior by applying instancing strategy and verifying reduced frame-time pressure in profiler captures.
- Standardized experiment notes and evidence bundles to make rendering optimization outcomes reproducible.

Tech: Unity, URP, C#, Unity Profiler

EXPERIENCE

Independent Unity Rendering Performance Engineer | 2024–Present

- Optimized rendering optimization hotspots with GPU/CPU bottleneck analysis, improving frame pacing and frame timing consistency.
- Profiled overdraw, MSAA, and submission patterns using Unity Profiler and Frame Debugger to prioritize fixes by measurable impact.
- Designed deterministic validation passes to prove improvements in draw calls, render thread behavior, and GPU RenderLoop cost.

Senior Unity Engineer (Mobile + Real-Time Rendering) | 2014–2024

- Improved scene-level rendering performance by reducing avoidable draw calls and improving batching/instancing strategy.
- Diagnosed CPU main thread and render thread hotspots with CPU/GPU profiling workflows and reproducible benchmark scenes.
- Shipped performance-sensitive mobile systems with guardrails for stability, memory/GC behavior, and regression prevention.

EDUCATION

Formal education details available upon request.